Freight rate prediction

Spotter · Machine Learning Engineer assessment

Denis Bakin

4 October 2026

1 Summary

The task is to predict `posted_rate`, the price in US dollars of one truck load, for 12,000 loads in November and December 2025, using 48,000 labelled loads from January to October 2025. The evaluation metric is not disclosed. The loads to predict lie entirely after the labels, so every development split moves forward in time: two early folds make every modelling choice, and a later fold that no choice ever saw gives the reported score (Section 3).

- **Model.** The log rate per mile is split into a market level $L(\text{date})$, a four-term linear model that can extrapolate, and a CatBoost model of the cross-sectional structure that takes L as its baseline (Section 4). Plain CatBoost cannot carry the rising price level two months ahead; the decomposition fixes that.
- **Result.** On the F3 holdout (September–October) the final model has an MAE of **$37.5 on clean rows (MAPE 1.61%)**, against $74.1 for a level-aware lane-median yardstick and $57.5 (2.35%) for CatBoost with a time coordinate. On all rows, corrupted labels included, the MAE is $93.5 (Section 5).
- **December chart.** $832–$884 for the fixed lane: a weekly wave from the market index (≈ $19), a gentle upward trend and a quarter-end ramp to $884 on 31 December (Section 6).

2 Data and quality issues

Each load has a lane (pickup and delivery city with coordinates, and the distance), an equipment type (Dry Van, Reefer or Flatbed), a weight, a date, a daily market-wide `market_index` and a weak per-load `quote_signal`. The findings from the exploratory analysis that shape the design are:

- **Out-of-time target.** Labels end on 31 October; the validation loads are all in November and December. Random splits would leak the future.
- **Unseen cities.** Eight cities, carrying 12.1% of the validation rows, never occur in training. The model therefore needs lane geometry (coordinates, distance), not only city identifiers.
- **Market signal.** `market_index` takes one market-wide value per day, has a weekly cycle (Thursday peak, Sunday trough) and correlates at 0.64 with the daily median rate per mile. `quote_signal` is weak (correlation about 0.1) and shows no sign of leakage.
- **Drift and noise.** At a similar `market_index`, the rate level rises by about 6% from January to October, with spikes at quarter ends. Within one lane, equipment and week, the rate per mile varies by only about 2.2% (coefficient of variation), so the achievable error is low.

Table 1 lists the data-quality problems and how each is handled. Only corrupted targets are dropped, and only from training; every repaired value gets an indicator column so the model can tell observed values from repaired ones. The validation inputs have the same issues (145 negative weights, 301 weights at the cap, 165 missing weights, 249 missing `market_index` values) and go through the same cleaning.

Table 1: Data-quality issues in the 48,000 labelled loads. Cap and floor counts are after the sign fix (13 and 1 of them were negative).

Issue	Rows	Handling
Corrupted target: rate ×2.5–5.5 or ×0.2–0.45	677 (1.4%)	Dropped (training, clean metrics)
Negative weight (a sign flip)	292 (0.6%)	Absolute value + flag
Weight at the 47,500 lb cap / 5,000 lb floor	1,204 / 32	Value kept + flag
Missing weight	300 (0.6%)	NaN kept (CatBoost-native) + flag
Missing `market_index`	374 (0.8%)	That day's mean + flag
Distorted coordinates, consistent per city	all	Used as internal geometry

Finding corrupted targets. Each load's rate per mile is compared with a robust reference, a hierarchical median (lane × equipment, then lane, then equipment × distance band) fitted on each fold's training rows only. The absolute log ratio is bimodal: in every fold's training and test windows, genuine loads stay below 0.23 and corrupted ones start above 0.73. The threshold $\log 1.6 = 0.47$ sits in this empty gap, so any value inside it flags the same rows.

3 Validation and split approach

Why forward-time only. The loads to predict come from the two months after the last label. A random or k-fold split would let the model see later market states while it is scored on earlier ones, and would overstate accuracy. Every fold below therefore trains on the past and tests on the two months that follow, the same situation as the real task (Figure 1).

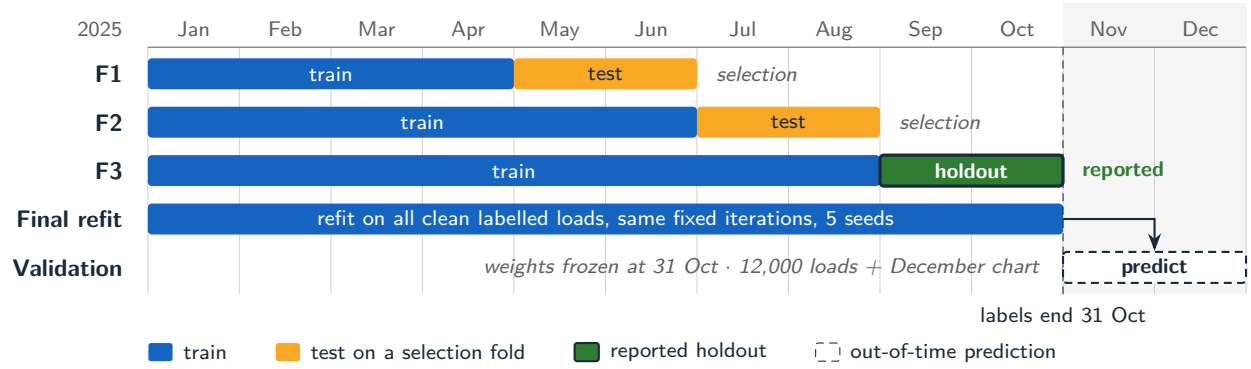


Figure 1: Expanding forward-time folds with two-month test windows. F1 and F2 make every modelling choice; F3 is the reported holdout; the final model is refit on all clean labelled loads and predicts November and December.

Folds. All folds train from 1 January. F1 trains on January–April and tests on May–June, F2 trains on January–June and tests on July–August, and F3 trains on January–August and tests on September–October. The final model trains on January–October and predicts the 12,000 November–December loads and the December chart.

Selection versus reporting. Every choice (the model design, the feature set, the hyperparameters) is made on the mean of F1 and F2. F3 is scored once with the final choice. The rule had a visible

cost: a variant that keeps a quote-signal feature scored \$2.6 better on F3 but \$4.4 worse on F1+F2 (\$55.0 against \$50.6), so it was not chosen.

Fixed iterations, several seeds. The number of boosting iterations (2,000) is part of the configuration and stays fixed when a model is refit; early stopping on a reported fold would quietly tune on it. CatBoost results depend on the seed, so each CatBoost model in the comparison (Table 3) runs with three seeds, reported as mean ± sd. The sd is \$1.9–3.0 for CatBoost with a time coordinate and \$0.1–0.5 for the final design.

Cold-start slice. To measure the unseen-city case on F3, 8 cities are drawn at random among those present in both windows, and every training row that touches one of them is removed (7,207 and 9,158 rows for the two draws). Test loads touching a held-out city are scored separately from the rest.

Metrics. Spotter has not disclosed its metric, so several are reported. The primary one is the mean absolute error in dollars (MAE) on clean rows, without the corrupted targets of Table 1; MAPE, MAE on all rows and RMSE come alongside. Two diagnostics support the analysis: the *month bias*, $\text{median}(\hat{y}/y) - 1$ per test month, and the *day-debiased MAE*, the clean MAE after removing each test day's median log error. The latter isolates the trees' cross-sectional error from the level's day-to-day error; it is used for tuning only.

4 Model

4.1 Plain trees miss the second month

A CatBoost model of the log rate per mile with lane, load, market and calendar features predicts the first month of each test window well (MAPE 1.0% in May, 1.2% in July), but under-predicts the second month by up to 6.6% (Table 2, left). The error sits in the daily price level. Trees cannot extrapolate beyond the target range they have seen, and the price level rises through the year. Features that change from month to month (`quote_signal`'s monthly level, days to the quarter end, trailing market averages) act as month proxies and let the trees reuse older, lower levels. Adding a time coordinate helps on F3 (MAPE 2.35%) but not reliably: on F1 the second month is still 7.2% low.

Table 2: Median bias in % by test month, $\text{median}(\hat{y}/y) - 1$ on clean rows. Plain CatBoost: one seed; the others: mean over three seeds.

		plain CatBoost		CatBoost + time t		L + CatBoost (final)	
Fold	Months	1st	2nd	1st	2nd	1st	2nd
F1	May / Jun	−0.3	**−6.6**	−0.5	**−7.2**	−1.1	−2.1
F2	Jul / Aug	−0.4	−2.8	−0.3	−1.0	+0.5	+2.6
F3	Sep / Oct	−2.5	**−6.5**	−1.5	−2.7	−0.7	−1.1

4.2 Decomposition: market level plus cross-section

The model splits the log rate per mile into a daily market level and a cross-sectional part:

$$\log \frac{\text{rate}}{\text{distance}} = L(d) + f(\mathbf{x}), \qquad L(d) = -0.1584 + 0.1192\,\bar{m}(d) + 0.000168\,t(d) + 0.0377\,q(d),$$

where d is the date, $\bar{m}(d)$ the day's mean `market_index` over all loads, $t(d)$ the number of days since 1 January 2025, and $q(d)$ a quarter-end ramp: zero outside March, June, September and December and up to day 15, then linear to 1 on the month's last day. The coefficients above are the final fit

on all of January to October (weighted $R^2 = 0.959$ over 304 days). A rise of 0.1 in the day's market index adds 1.2% to the level, the trend adds 0.51% per 30 days, and the ramp adds 3.85% on the last day of a quarter.

L is fitted once per training window: the daily mean of log(rpm/reference rpm) over the clean training loads is regressed on the four terms by least squares, weighted by the number of loads per day. CatBoost then fits the log rate per mile with L as its baseline, so the trees learn only what the level does not explain: lane structure and per-load deviations. Every term of L is known in advance. The day's market index is an input for every November and December load, and t and q are calendar functions, so L extrapolates where trees cannot (Figure 2). Both calendar terms matter: without the trend, F3 MAE rises to \$83.5; without the ramp, F2 MAE rises to \$87.5, because the trend absorbs the June spike and overshoots in July and August. A second backfitting round (refit L on the tree residuals) did not help on F1+F2 (\$58.0 against \$55.0), so one round is used.

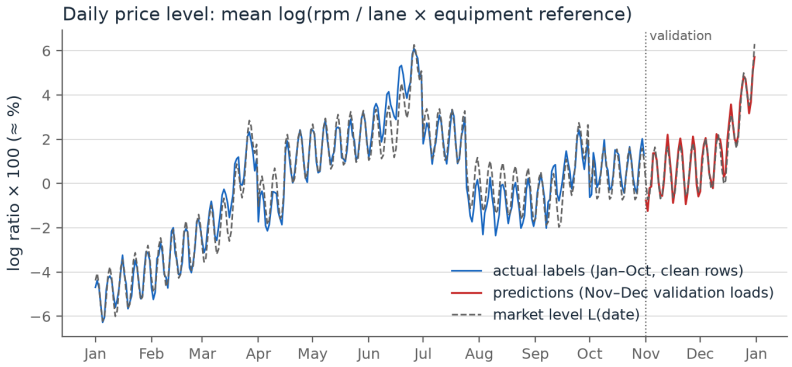


Figure 2: Daily price level, the mean of log(rpm/lane × equipment reference) per day: clean January–October labels (blue), the fitted L (dashed) and the November–December predictions (red). The weekly wave follows the market index; the steps at the ends of March, June and September are the quarter-end ramp, which returns in late December.

4.3 The tree part

f is a CatBoost regressor (RMSE loss on the log rate per mile, depth 6, learning rate 0.05, 2,000 iterations: the library defaults, see Section 5.3) with 24 features: pickup and delivery city, lane and equipment (categorical); coordinates, crow-fly distance, bearing, latitude and longitude deltas, road distance, its logarithm and circuity (road over crow-fly distance); weight and its four flags; the load's `market_index` minus the day's mean, with its missing flag; weekday and day of month. No time coordinate, month or trailing average enters the trees; the level belongs to L. `quote_signal` is dropped. The relation between a load's quote signal and its price changes sign with the month's average quote signal, so it gives no reliable direction for November and December, and it lost the F1+F2 selection (\$55.0 with it, \$50.6 without).

The final model is fitted on all 47,323 clean January–October loads (677 corrupted labels excluded). L is fitted once, five CatBoost seeds are averaged on the log scale, and a prediction is distance $\cdot$ $\exp(L(d) + \hat{f}(\mathbf{x}))$.

5 Results

5.1 Model comparison

On the F3 holdout the final model halves the yardstick's error and cuts that of CatBoost with a time coordinate by 35% (Table 3). The 144 corrupted F3 test labels (1.5% of the window) add

about \$56 to every model's MAE and put RMSE on all rows at \$630–637 for every model, so RMSE on all rows cannot separate the models. The 5-seed ensemble is \$0.19 better than a single seed (\$37.38 ± 0.13), the expected small variance reduction.

With L carrying the level, the second test month is no longer systematically low (Table 2, right). What remains is L's own extrapolation error: up to 2.6% on F1 and F2, about 1% on F3. Removing each test day's median error lowers the MAE from \$50.6 to \$32.9 on F1+F2 and from \$37.5 to \$32.6 on F3; the rest is cross-sectional error.

Table 3: MAE in dollars on clean rows unless marked "all rows"; mean ± sd over n CatBoost seeds. F1+F2 decides; F3 is scored once.

		selection folds			F3 holdout (Sep–Oct)		
Model	n	F1	F2	F1+F2	clean	all rows	MAPE
Lane median × distance (yardstick)	1	148.8	83.8	116.3	76.2	131.7	3.21%
Lane median × e^L	1	82.2	83.3	82.7	74.1	129.6	3.13%
CatBoost + time t	3	83.1 ± 2.8	67.1 ± 3.0	75.1	57.5 ± 1.9	113.3	2.35%
L + CatBoost, with quote signal	3	49.4 ± 0.3	60.6 ± 0.5	55.0	34.8 ± 0.1	90.9	1.50%
L + CatBoost (chosen)	3	50.7 ± 0.2	50.5 ± 0.4	**50.6**	**37.5 ± 0.1**	93.5	1.61%
chosen, 5-seed ensemble	5	–	–	–	37.2	93.2	1.60%

5.2 Where the errors are

The error is a nearly flat 1.5–1.85% of the rate across distance, so the dollar MAE grows with distance. Flatbed is the hardest equipment type. Removing a city from training costs about \$6 of MAE on its loads (\$40.3 against \$34.4): the coordinates and the distance carry most of what the city identity would.

Table 4: F3 slices, clean rows, chosen model (mean over three seeds); MAE in dollars, bias is the median. *Cold start: the variant with the quote-signal feature, one seed, mean of two draws of 8 held-out cities.

Equipment	n	MAE	MAPE	bias
Dry Van	5,275	33.4	1.52%	−0.73%
Flatbed	1,746	47.6	1.97%	−1.46%
Reefer	2,358	39.1	1.55%	−1.00%
Cold-start slice*				
held-out cities		40.3	1.70%	
other cities		34.4	1.48%	

Distance (mi)	n	MAE	MAPE
0–150	184	6.0	1.85%
150–300	585	11.5	1.84%
300–500	1,285	16.4	1.66%
500–750	1,476	23.3	1.63%
750–1,000	1,452	30.0	1.56%
1,000–1,500	1,735	41.3	1.61%
1,500–2,000	1,149	55.6	1.55%
2,000–2,500	958	67.2	1.53%
2,500+	555	79.9	1.48%

5.3 Tuning changed little

A feature-family ablation came first, scored by the F1+F2 day-debiased MAE (\$32.92 with all 24 features): masking weight costs \$26.6 and the coordinates \$1.9, and every other family moves it by less than \$0.5. No family met the drop rule (a gain above \$1), so all 24 features stay. Optuna (TPE sampler, median pruner, 75-minute budget) then ran 51 trials on the same objective (32 complete, 18 pruned, 1 lost to a power outage), over depth, learning rate, iterations, L2 and random strength, borders, bootstrap and grow policy. The defaults and the five best trials were re-run with three seeds (Table 5). The best trial gains \$0.34 day-debiased and \$0.10 in plain MAE, inside the \$0.5 tie

rule, and fits about 3× slower, so the defaults were kept. The decomposition moved the F3 MAPE from 2.35% to 1.61%; the hyperparameters barely moved the selection metric.

Table 5: Re-evaluation on F1+F2 with seeds 0–2 (the three best trials and the defaults): MAE in dollars, mean ± sd over seeds, and fit time per fold.

Candidate	depth	learning rate	iterations	bootstrap	day-debiased	plain clean	fit (s)
trial 36	8	0.021	3,500	Bernoulli	**32.57 ± 0.06**	50.53	65.5
trial 41	8	0.021	3,100	Bernoulli	32.57 ± 0.05	50.58	64.7
trial 46	8	0.026	2,400	Bernoulli	32.63 ± 0.07	50.61	44.4
defaults (kept)	6	0.05	2,000	MVS	32.91 ± 0.05	50.63	**23.7**

5.4 TabPack: a tabular deep-learning check

To test whether the cross-sectional learner is the bottleneck, CatBoost was replaced by TabPack,[1] a packed ensemble of 64 MLPs with randomly sampled hyperparameters, trained in parallel, with periodic numerical embeddings, the Muon optimiser and greedy online ensemble selection on a validation split. It ran in the paper's configuration with three seeds on the same folds; its validation split is the last four weeks of each training window, which its networks do not train on. It was trained in CatBoost's place on $\log \mathrm{rpm} - L(d)$, standalone on $\log \mathrm{rpm}$, and with lane identity added (Table 6). The 36 runs took 38 minutes on a rented RTX 5090.

Table 6: TabPack against the chosen model: MAE in dollars, mean ± sd over three seeds.

	selection folds, clean			F3 holdout	
Model	F1	F2	F1+F2	clean	day-debiased
L + **CatBoost (chosen)**	50.75 ± 0.23	50.52 ± 0.37	**50.63**	**37.45 ± 0.10**	32.56
TabPack, no L	39.65 ± 2.66	66.74 ± 0.66	53.20	60.36 ± 3.66	29.23
L + TabPack	45.73 ± 0.45	67.42 ± 2.17	56.57	39.89 ± 0.21	31.86
L + TabPack, lane one-hot	45.56 ± 0.20	66.57 ± 2.26	56.06	40.13 ± 0.20	31.69

Without L, TabPack fails like plain trees. It wins F1 but is \$16–23 worse than the chosen model on F2 and F3 (F3 MAPE 2.48%), with its second F3 month 3.0% too low. Its F3 day-debiased MAE is the lowest, but on F2 it is \$67.33: learning the level from a time coordinate breaks the cross-section too. The level model is what makes a tabular model stable.

In CatBoost's slot, the cross-section is better and the level is worse. Day-debiased, L + TabPack beats CatBoost on every fold (by \$0.26, \$0.70 and \$0.70), but its F2 month bias is +1.67% and +3.60% (CatBoost: +0.52% and +2.55%), so its plain MAE is worse; plausibly (not tested) because its networks never see the last four training weeks. Lane identity, top-155 or one-hot, moves F1+F2 by \$0.24–0.51, inside the F2 seed sd (\$2.2–3.4).

Blend and gate. A blend $w \log \hat{y}_{\mathrm{CB}} + (1-w) \log \hat{y}_{\mathrm{TP}}$ was to be chosen on F1+F2 clean MAE and to replace CatBoost only with a gain above \$0.5. The clean MAE falls monotonically to $w = 1$, CatBoost alone: the gain is \$0.00, the gate fails, and F3 at the chosen w is CatBoost's \$37.30 (three-seed log-mean). The day-debiased optimum, $w = 0.4$, gains \$0.76 on F1+F2 and on F3 (day-block bootstrap 95% intervals exclude zero): real, but too small to survive TabPack's level offset. A post-hoc blend with the standalone model (\$42.52 on F1+F2 at $w = 0.6$) was not pre-specified and scores \$43.20 on F3: two opposite level errors cancel on F2. The submission is unchanged;

[1] Y. Gorishniy, A. Kotelnikov, I. Rubachev, A. Babenko. *TabPack: Efficient Hyperparameter Ensembles for Tabular Deep Learning.* ICML 2026; arXiv:2607.05380.

TabPack's own November–December predictions sit 0.42% above CatBoost's. A stronger learner was worth about $1; the level decomposition was worth about $20 on F3.

6 December prediction chart

The December chart fixes the lane (Lexington to Fort Wayne, 360 miles, Dry Van, 32,000 lb) and asks for one prediction per day; only the date changes. Figure 3 is the chart produced by the provided `score.py`.

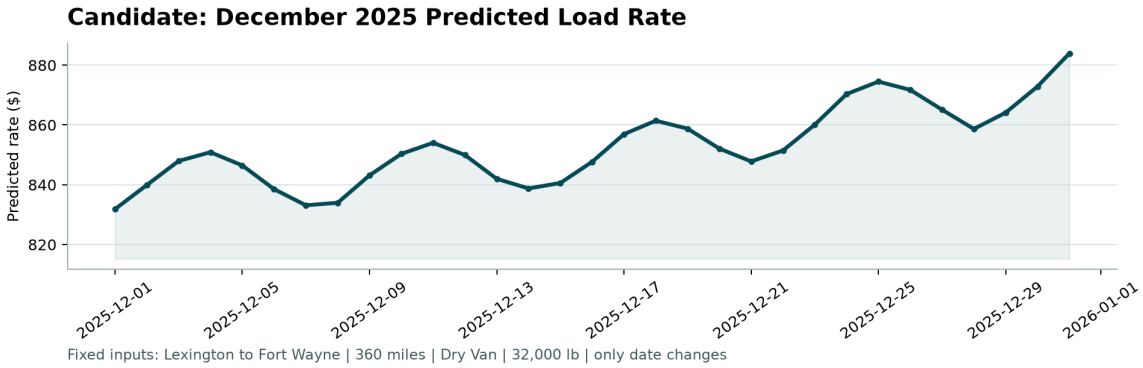


Figure 3: Predicted rate for the fixed December lane, one prediction per day, as drawn by `score.py`.

Inputs for December. The chart file has no `market_index` or `quote_signal` column. Each December day gets that day's mean `market_index` over the loads in `validation.csv`. These are model inputs, not labels: `validation.csv` contains no rates. The model weights stay frozen at 31 October while the inference inputs are fresh, exactly as in production, where the day's market index is known when a load is quoted. No November or December label enters the model at any point.

Reading the shape. With the lane, weight and distance fixed, the shape comes from L: a weekly wave from the market index (Thursday +1.2% and Sunday −1.1% against a neutral market, an amplitude of about $19), the trend (+0.51% per 30 days) and the quarter-end ramp from 16 December (+3.85% on 31 December). The tree part adds little; its weekday and day-of-month effects span $10.9 over the month. The curve runs from $831.84 on Monday 1 December to $883.92 on Wednesday 31 December, with a mean of $852.88.

Neutral-market comparison. Figure 4 repeats the chart with every December day's market index set to the December average. The two curves stay within $11.4 (1.3%) of each other and have near-equal means ($852.88 and $852.84). The market input sets the day-to-day shape; the trend and the ramp set the rise, and on the neutral curve 25–31 December is 2.9% above 1–15 December.

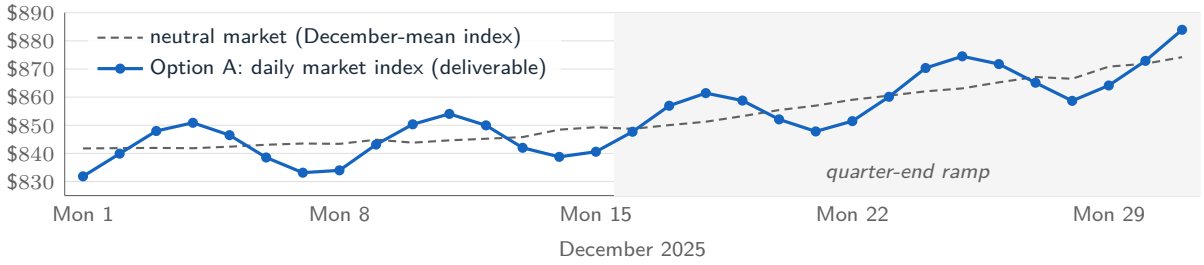


Figure 4: The chart lane under the daily market index (the deliverable) and under a neutral market.

Against the lane's history. The chart sits 5.7% above the lane's September–October median scaled to 360 miles ($807.01, four loads), mostly because those loads are lighter (median 24,665 lb); at that weight the chart's mean would be $825.31.

7 Limitations and next steps

- **Holidays are not modelled.** Thanksgiving and Christmas fall inside the prediction window but outside the training data, so the model has no estimate of their effect.
- **The trend is extrapolated.** L carries +0.51% per 30 days past 31 October. Had the drift stopped that day, the December predictions would be 0.5% (1 December) to 1.0% (31 December) too high, about 0.8% in mid-December: a bounded risk, below the error level.
- **β_{mi} and β_t trade off over short windows.** Across the folds, β_{mi} ranges from 0.088 to 0.124 and β_t from 1.4 to 2.5×10^{-4} per day, in opposite directions. L's out-of-sample level error reached −2.2% in June (F1) and +2.7% in August (F2): the main remaining error source.
- **The weekly wave is slightly too large:** 2.29% predicted against 1.93% in the labels, because L uses a single market-index slope.
- **Flatbed** is the weakest segment (F3 MAPE 1.97%, median bias −1.46%).
- **The metric is unknown.** Validation labels corrupted at the training rate (1.4%) would inflate any metric, squared errors most; the model targets the typical price.
- **Next steps:** holiday priors; refit TabPack on its training and validation weeks with a fixed epoch count, which removes its four-week holdout, or use it only in a blend on a level-corrected target (Section 5.4); a monotone constraint on weight in the trees.

8 Reproducibility

The code is a Python package (`src/freight/`) with thin scripts and a Makefile. The organisers' CSV files are not redistributed; place them in `data/` under the names as distributed, with hyphens: `train-test.csv`, `validation.csv`, `december-chart-inputs.csv`, `validation-predictions-template.csv`.

```
make setup      # Python 3.12 venv in .venv + requirements + package
make all        # tests, train, predict, score, sanity (about 6 min)
make research   # backtests, ablation, Optuna (a few hours)
make tabpack    # TabPack check (TabPack clone + its venv; GPU recommended)
make notebook   # re-run notebooks/analysis.ipynb (about 10 s)
make report     # this report: reports/_output/report.pdf
make deck       # the slides:  reports/_output/deck.pdf
```

`make all` runs the 59 tests, fits the final model, writes `validation_predictions.csv` and `december_chart_predictions.csv`, runs the organisers' `score.py` on both (which draws `scorer_results/candidate_december.png`) and writes sanity checks to `artifacts/final/`. It reads the committed `artifacts/tuning/best_params.json`, so it does not need `make research`. Seeds are fixed; versions (Python 3.12.3, CatBoost 1.2.10, pandas 2.3.3, NumPy 2.5.3, Optuna 4.9.0) are recorded in `artifacts/final/model_card.json`.

`make tabpack` reruns the TabPack check (Section 5.4). It needs the TabPack clone in `tabpack/` (not redistributed) with its own venv, `tabpack/.venv`, beside `.venv`, and ideally a GPU (38 minutes on an RTX 5090; a CPU run is far slower). Its outputs are committed in `artifacts/tabpack/`; the setup and the remote-GPU recipe are in `scripts/tabpack/README.md`.